picoDAW: a web-based sequencer/synthesizer/sampler

Tim Fan (SID: 550790019)
fanchenhao2206
repo: https://github.com/fanchenhao2206/elec5305-project-550790019
site: https://fanchenhao2206.github.io/elec5305-project-550790019

I. Project Overview

This project aims to develop a music sequencer with a synthesizer and sampler as available instruments, for any device capable of running a modern web browser supporting WebAudio, using the emscripten compiler toolchain that turns C++ DSP code into WebAssembly.

The goal is to make music production more accessible; pretty much any computer—whether that be a smartphone, laptop, or even a single-board computer—can run a modern web browser that supports WebAudio[1]. This is important because anyone should be able to play with and make music, without needing to install and learn a comprehensive desktop-based music production application designed for professionals and not the general public.

II. Definitions

Before moving on, I want to briefly describe what I mean by a sequencer/synthesizer/sampler, and also what a digital audio workstation—aka DAW—is.

A. Synthesizer

What I mean by a synthesizer, is an instrument that generates and plays a sound on demand. For example, a basic sawtooth synthesizer generates a sawtooth wave sound on demand, its frequency—aka pitch—depending on which key was played on the input keyboard controller connected to it. Pressing the key corresponding to the pitch A5, will result in the synthesizer synthesizing and playing a sawtooth wave sound at 880Hz.

B. Sampler

A sampler, in my understanding, is actually very similar, except that instead of generating a new sound, it will take an existing sound—e.g. the wave format recording of a real-life snare drum being hit—and play it back on demand; possibly also modifying or processing it before playing it back. For example, pressing one key might play back the snare drum recording in its original form, literally re-playing the original wave recording. While pressing another key might play back the recording at half its sample rate, resulting in the same sound but lowered one octave.

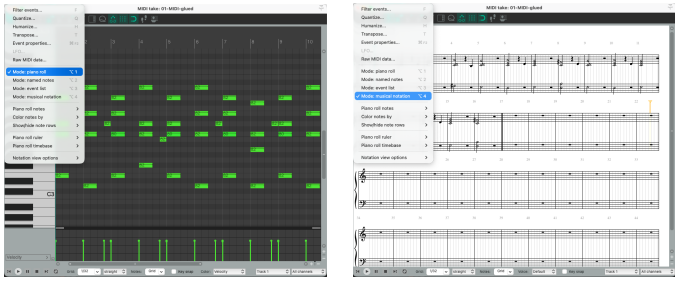
[1]https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API#browser_compatibility

C. Sequencer

At the core of both these instruments are their respective input controllers; typically used by the human body, whether that be through hitting it, blowing air into it, stepping on it, etc. The sequencer then, in my understanding, gets its name from the idea that an instrument can be played not by hitting it directly, but by providing it a sequence of instructions on what keys the instrument should press for itself, and when and in what order.

An example of such an instrument is the player piano[2], which reads the desired keys to be pressed from a sheet of paper, written by a human who punches holes in the paper corresponding to the keys they want the instrument to press, and when they should be pressed. In essense, like how early computers were programmed, through punch cards. Presumably, because the sheets of player piano paper could get very long, they would be rolled up, giving them their name; piano rolls[3].

A sequencer then, in my understanding, is the machine a human would use to write the program—e.g. a piano roll—that will then be loaded by an instrument—e.g. a player piano—that can understand this program. Perhaps the most common type of sequencer, fittingly, is based off the piano roll. In REAPER[4] for example, the "piano roll" mode of its sequencer is the default mode (fig. 1a). Other modes include the "musical notation" mode, which is based off the Western Art or classical music tradition of sheet music (fig. 1b).



(a) Piano Roll (b) Musical Notation

Fig. 1: The modes of REAPER's MIDI sequencer

[2]https://en.wikipedia.org/wiki/Player_piano
[3]https://en.wikipedia.org/wiki/Piano_roll
[4]https://www.reaper.fm

D. Digital audio workstation

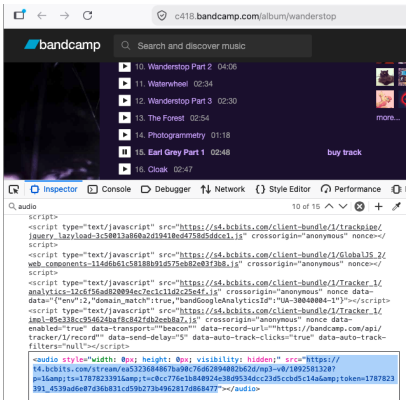
A typical DAW then, is an application that combines all of the above—sequencer/synthesizer/sampler—alongside many other features useful for professionals and experienced hobbyists. For example, a DAW like the aforementioned REAPER will also allow you to record audio through it; the recording can then of course be played back though its sampler. Note that because the sampler is also an instrument, the timing of playback can also be sequenced through the included sequencer. And, though it comes with pre-installed instruments, third-party instruments can also be used with its sequencer, through integration with software instrument standards like the VST[5].

III. Background and Motivation

A. WebAudio

Despite its relatively recent history, with the first publication of a working draft in only 2011[6], the WebAudio API has seen extensive development over the course of a decade, eventually receiving an endorsement from the W3C in 2021 as a Recommendation[7]. Notably, its primary `AudioContext` interface has been fully supported by most major web browsers since at least 2015[1], with the only exception being Safari which implemented support in 2021.

Of course, audio has existed for a much longer time than that on the internet. As Boris Smus describes in their brief history of audio on the web[1], initially that was through the Flash external plugin for browsers, and then later through the HTML5 `<audio>` element which was instead natively supported by the web browser itself. In fact, it's this `<audio>` element that allows music streaming websites like Bandcamp to preview an artist's music through the web browser (fig. 2).




```
Input ... from '1092581320.mp3' Duration:
  00:02:48.00, ..., bitrate: 236 kb/s
```

Fig. 2: Bandcamp allows artists to offer their listeners a compressed and lower bitrate MP3 preview of their tracks, played in the browser through the HTML5 `<audio>` element

But one thing Flash's plugin did for web audio that HTML5's `<audio>` element doesn't do, is allow for interactive audio. For example, a sequencer/synthesizer/sampler like the one this project will aim to build, would require a predictable amount of latency between a person pressing a key on their keyboard and the corresponding sound being played through their speakers, something that the `<audio>` element just wasn't designed for.

B. Web-based Synthesizers

That is the kind of problem the WebAudio API attempts to solve, and why it has been possible since then for synthesizers and other real time instruments to be built for the web. For example Steven Goldberg's emulation of the Juno-106 synthesizer[8] built for the web in 2015, or Takamitsu Endo's collection of synthesizers[9] built for the web starting in the 2020's, both using the aforementioned `AudioContext` interface as the backbone of their audio framework—like in Takamitsu Endo's `common/wave.js`[10]—and many other examples; like the web synthesizers available at playtronica[11].

There are also examples of projects for the web that combine a synthesizer with an integrated sequencer, in the same vein as what this project aims to do. One notable example is certainly André Michelle's opendaw[12], containing a sequencer, synthesizers, samplers, and more; everything you would expect from a conventional desktop-based DAW. However, even its interface strongly resembles a conventional desktop-based DAW; it also requires a desktop browser.

IV. Proposed Methodology

That is why through this project, the author argues the need for a stripped-down DAW—consisting of just a sequencer/synthesizer/sampler; hence the name picoDAW—that is accessible because (i) it is built for the web, (ii) has a simplified interface that is not like a desktop DAW, and (iii) has a minimal feature set that can still provide some expression.

Of course, this kind of stripped-down DAW is certainly not a new product category. Examples include AKAI's MPC[13] or Elektron's Digitakt[14] series of hardware music production workstations. The product this project will take primary inspiration though, is a software one called the KORG DS-10[15]—released in 2008 for the Nintendo DS, a touchscreen and handheld video game console—where the hardware knobs and patches of an expensive groovebox have been translated into software controls and buttons on the console's 3" touchscreen.

It is a good point now to mention that in the pursuit of a simpler interface than traditional desktop-based DAWs, these workstations opt to use a simpler type of sequencer called a step sequencer. Most notable is that instead of allowing arbitrary timing of key presses to be sequenced—for example, exactly 4.2 seconds after the previous note—the step sequencer

[5]https://en.wikipedia.org/wiki/Virtual_Studio_Technology
[6]https://www.w3.org/TR/2011/WD-webaudio-20111215/
[7]https://www.w3.org/TR/webaudio-1.0/
[8]https://github.com/stevengoldberg/juno106
[9]https://github.com/ryukau/UhhyouWebSynthesizers
[10]https://github.com/ryukau/UhhyouWebSynthesizers/blob/main/common/wave.js
[11]https://synth.playtronica.com/
[12]https://opendaw.studio
[13]https://www.akaipro.com/products/collections/mpc/
[14]https://www.elektron.se/explore/digitakt-ii
[15]https://archive.org/details/korg-ds-10-manual/korg_ds-10_manual/

enforces a strict rhythm; integer intervals of the project's base timestep. For example, if it was 1 second—aka 60 beats per minute—then the idea is that keys can be sequenced only on 1, 1/2, or 1/4 second intervals; relative to the base timestep (so, 120bpm would mean 0.5, 0.25, and 0.125 second intervals). See fig. 3 for a photo of the DS-10's step sequencer. Futhermore, the step sequencer will be monophonic; only one key can be pressed on each step.

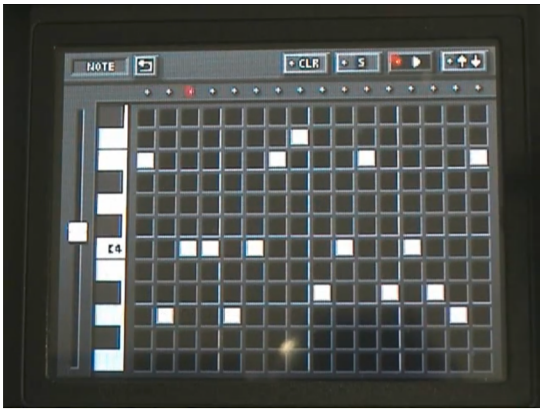


Fig. 3: The KORG DS-10's monophonic step sequencer; photo taken from https://www.youtube.com/watch?v=Usjy626gQYQ

Thus, implementing this project requires not just writing DSP code for e.g. real-time filtering or waveform shaping, but also designing and building a user interface to go with it. To ease development, this project will use the iPlug2[16] library, a cross-platform C++ audio plugin framework; one platform being the web browser. This means both the backend DSP code and frontend UI code can be written in C++, and then compiled into JavaScript through emscripten[17] for running in the browser.

This leaves room to focus a better portion of development time on the backend, researching and implementing the actual DSP and audio-processing algorithms that will power the picoDAW. Initial prototyping of ideas will be in MATLAB, implementation will be through C++ compiled to WebAssembly, and testing will be through CMake. DSP and audio effect related resources that could be of use include the Digital Audio Effects book by Udo Zölzer[2], and The Art of VA (Virtual Analog) Filter Design by Vadim Zavalishin[3].

As mentioned, iPlug2 provides a library for building frontend interfaces using C++. Specifically, one of the options is the nanovg[18] graphics library, itself a wrapper around OpenGL API calls; what ultimately draw graphics onto the screen. The web browser, however, does not understand it specifically. Instead it understands the WebGL API—intentionally designed to conform closely with OpenGL[19]—and so emscripten also works to translates nanovg's OpenGL calls written in C++ to equivalent WebGL calls written in JavaScript, allowing UI built using iPlug2 to work in the browser.

[16]https://github.com/iplug2/iplug2
[17]https://emscripten.org/
[18]https://github.com/memononen/nanovg
[19]https://registry.khronos.org/webgl/specs/latest/1.0

V. Expected Outcomes

Of course, the ultimate deliverable will be a working prototype of picoDAW deployed onto the provided GitHub project site. Given the complexity of a DAW however—even a stripped down one, only a sequencer/synthesizer/sampler—the core mantra behind the project will be to keep it simple, stupid.

This means, the UI will be basic without the polish of the actual DS-10 it'll take inspiration from. Furthermore, though audio effects like real-time reverb or delay, and features like a semi-modular patchbay, are incredibly useful in modern music creation, they will not be prioritised over a working prototype.

What is a non-negotiable though, is the inclusion of a low pass and high pass filter for each instrument, and an envelope to control its cutoff frequency; alongside a second envelope for volume. In summary, the expected features will be a:

1) Synthesizer and sampler as instruments
   - 4 Synthesizers; each has sine, saw, triangle, square, and noise as possible oscillator options
   - 2 Samplers; each can load a `.wav` sample from 0.1 to 1 second, resampled to 48kHz on upload
2) Step sequencer with 16 steps. At a default 120bpm, this means only 8 seconds of audio, played in a loop
   - The 6 instruments will each have their own
   - The sequencer will have two screens
     - one for sequencing what key is pressed and when
     - one for how long that key should be pressed

     termed Note and Gate respectively on the DS-10
   - Changes to pattern during play will reflect on loop
3) Filter with adjustable cutoff and peak/resonance, toggled between lowpass and highpass.
   - The 6 instruments will each have their own filter
4) ADSR[20] envelope
   - The 6 instruments will each have two envelopes
     - one for modulating the cutoff of its filter
     - one for modulating the volume of its output
5) Mixer
   - Basic; toggle mute or solo for each instrument

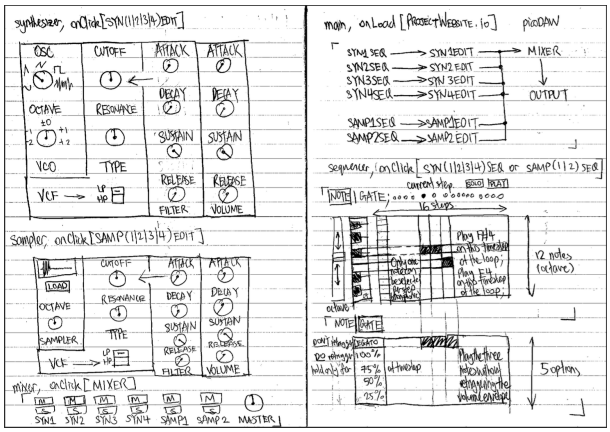


Fig. 4: Draft mockup of UI for picoDAW

[20]https://en.wikipedia.org/wiki/Envelope_(music)#ADSR

Stretch goals, in case development goes easier than expected, include: (i) Pattern sequencer, enabling more expressive song creation; build multiple patterns, and then sequence them together to create a song. (ii) Two oscillators per synthesizer—instead of just one—making it easier to create chords, since the pitch of the second oscillator could be tuned to a specific note above or below the first oscillator. (iii) Volume control in mixer for each instrument, and basic effects—reverb, delay, and EQ—with routing done through the mixer. See fig. 4 for a draft of how the UI could work.

VI. Timeline

Week	Task
1–4	Considered project topic; research and literature review
5–6	Implement sequencer to trigger included sine synth wrt. volume envelope; on notes and following gate values
7–8	Setup testing framework; implement low-pass/high-pass filters with envelopes on their cutoff frequencies
9–10	Implement additional oscillators, sampler, and mixer; refactor system to maximise code reuse
11–13	Report writing; bug fixes, implementation of extra features if stretch goals are met, prototype cleanup

References

[1] B. Smus, *Web audio API: advanced sound for games and interactive apps*. O'Reilly Media, Inc., 2013.
[2] U. Zölzer, *DAFX: Digital Audio Effects*. Wiley, 2011.
[3] V. Zavalishin, *The Art of VA Filter Design (rev. 2.1.2)*, 2020.